# ADH Requirements Specialist Agent

## 1. Purpose & Scope

The **ADH Requirements** specialist agent is the first phase in the ADH Pilot pipeline. It merges all input sources (BRD files, meeting transcripts, chat messages, and developer clarifications) into a single, structured `requirements.md` document that serves as the authoritative source of truth for all downstream phases.

This spec tells a developer exactly how to build `.github/agents/adh-requirements.agent.md`.

**What this agent does:**

- Reads BRD file(s), transcripts and context provided by the master (ADH Pilot)
- Extracts and merges information across all sources
- Identifies conflicts, ambiguities, and missing information
- Writes a structured `requirements.md` following the template at `adh-templates/artifacts/requirements.template.md`
- Determines which downstream phases are required (silver? gold? SQL views? tests?)
- Returns structured `PHASE_RESULT` with validation status and questions (if any)

**What this agent does NOT do:**

- Never asks questions directly to the developer (surfaces them via QUESTIONS status to master)
- Never makes design decisions (that's Phase 02 - HLD)
- Never invents missing data (flags it as Open Item instead)
- Never edits files in-place (always writes new or overwrites)
- Never invokes other specialist agents (only the master can do that)

**Key design principle:** If information conflicts or is missing, this agent **flags it as an Open Item** (Section 11 of requirements.md) and returns `status: QUESTIONS`. Silent assumptions are forbidden.

## 2. Integration Context

### 2.1 Inputs (provided by master via delegation prompt)

The master agent (ADH Pilot) invokes this specialist by running `build_prompt.py` which assembles a fully-resolved delegation prompt containing:

| Input | Source | Format |
|---|---|---|
| **BRD file path(s)** | `.adh/workspace/input/{project}/` | One or more `.md`, `.txt`, or `.docx` files |
| **Transcript content** | Inline text or file path | Meeting notes, stakeholder conversations |
| **Chat context** | Prior developer messages in this session | Key clarifications, ad-hoc requirements |
| **Custodian config** | `.adh/custodians/{name}.config.yml` | Catalog prefix, KV scope, storage accounts |
| **Project metadata** | `project-tracker.json` | Project name, environment (dev/tst/stg/prd) |
| **Rework feedback** | `--rework "{text}"` (if Phase 01 is being retried) | What the developer wants changed |

**The specialist never reads these files directly.** The master assembles them and passes them as a single delegation prompt. The specialist processes the **content** provided.

#### 2.2 Output

Single file: `.adh/workspace/output/{custodian}/{project}/requirements.md`

**Folder structure the specialist writes to:**

```
.adh/workspace/output/{custodian}/{project}/
  requirements.md           ← Phase 01 output (this specialist writes this)
  hld.md                    ← Phase 02 (written by ADH HLD specialist)
  lld.md                    ← Phase 03 (written by ADH LLD specialist)
  configs/                  ← Phase 04 (written by ADH Code-Bronze specialist)
  jobs/                     ← Phase 04
  notebooks/                ← Phases 05, 06
  sql/                      ← Phase 07
  tests/                    ← Phase 08
  review-report.md          ← Phase 09
  MANIFEST.md               ← Phase 10

```

**The specialist only touches `requirements.md`.** All other files are out of scope.

#### 2.3 Tracker integration

The specialist updates the tracker at two points:

**On START (first action):**

```
py .github/skills/tracker-checkpoint/tracker.py update \
   --custodian {name} \
   --project {project} \
   --phase 01 \
   --status started

```

**On completion (after writing requirements.md):**

```
py .github/skills/tracker-checkpoint/tracker.py update \
   --custodian {name} \
   --project {project} \
   --phase 01 \
   --status done \
   --outputs requirements.md

```

**If FAIL or QUESTIONS:** Do NOT call `tracker.py update ... done`. Leave status as `started`. The master will reset the phase when retrying.

---

### 3. Deliverable

**One file:** `.github/agents/adh-requirements.agent.md`

```
---
name: "ADH Requirements"
description: "Phase 01 - Merges all input sources (BRD, chat,
transcripts) into a single structured requirements.md. Invoked by ADH
Pilot only."
model: claude-sonnet-4.5
tools:
  - vscode/readFile
  - vscode/writeFile
  - vscode/runTerminal
user-invocable: false
---

```

#### 3.1 Field notes

| Field | Value | Rationale |
|---|---|---|

| | | |
|---|---|---|
| `name` | `"ADH Requirements"` | Matches the Phase Overview table in DESIGN-PROPOSAL-V4.md |
| `model` | `claude-sonnet-4.5` | Sonnet for reasoning tasks: merging conflicting sources, identifying ambiguities, extracting implicit requirements |
| `vscode/readFile` | Yes | Reads BRD files if paths provided (though master typically inlines content) |
| `vscode/writeFile` | Yes | Writes `requirements.md` to `.adh/workspace/output/{custodian}/{project}/` |
| `vscode/runTerminal` | Yes | Runs `tracker.py` for phase tracking |
| `user-invocable` | `false` | **Critical:** Developer cannot open this agent directly. Only the master (ADH Pilot) can invoke it. |

**Security enforcement:** All specialist agents set `user-invocable: false`. The master sets `user-invocable: true`. This prevents developers from bypassing the orchestrated pipeline.

---

#### 4. Tool Grant (Least Privilege)

| Tool | Granted? | Purpose |
|---|---|---|

| | | |
|---|---|---|
| `vscode/readFile` | Yes | Read BRD file(s) if passed as paths; read prior requirements.md if rework |
| `vscode/writeFile` | Yes | Write `requirements.md` to `.adh/workspace/output/{custodian}/{project}/` |
| `vscode/runTerminal` | Yes | Execute `tracker.py update 01 started` and `tracker.py update 01 done` |
| `vscode/findFiles` | **No** | Master provides file paths; specialist doesn't search |
| `vscode/askQuestion` | **No** | Specialists never ask questions directly; use QUESTIONS status instead |
| `agent` | **No** | Specialists cannot invoke other specialists |

**Why no askQuestion?** The master is the single point of developer interaction. All questions are surfaced via the `PHASE_RESULT` block (see Section 7).

---

## 5. Logic Flow

#### 5.1 Phase guard (entry validation)

**On invocation, immediately:**

```
1. Check that .adh/tracker/{custodian}/{project}/project-tracker.json
exists
   → If missing: STOP with status=FAIL, message="Tracker not
initialized"

```

```
2. Read project-tracker.json
   → Confirm current_phase is "01"
   → If not "01": STOP with status=FAIL, message="Phase guard failed - expected 01, got {current_phase}"

```

**Why this guard?** Prevents out-of-order execution. The master should always set `current_phase` correctly, but the guard catches configuration bugs early.

#### 5.2 Execution steps

**STEP 0 - Update tracker to "started"**

```
py .github/skills/tracker-checkpoint/tracker.py update \
   --custodian {custodian} \
   --project {project} \
   --phase 01 \
   --status started

```

**STEP 1 - Collect all inputs**

Parse the delegation prompt provided by the master. It contains:

- **BRD content:** Either inline text (most common) or file paths to read
- **Transcript content:** Meeting notes, stakeholder conversations
- **Chat messages:** Developer clarifications from prior conversation turns
- **Rework feedback:** If `--rework` flag was passed, what needs to be fixed

Read every source fully. If file paths are provided, use `vscode/readFile` to load them.

**STEP 2 - Extract and merge across all required sections**

For each section of the requirements.md template (see Section 6), extract relevant content from all input sources. The canonical template has **core sections 0-11** (Section 0 is optional) plus optional sections 12-15.

**Conflict resolution rules:**

| Scenario | Action |
|---|---|
| **Sources agree** | Use the agreed value |
| **Sources conflict** | Add to Section 11 (Open Items) with both versions; status → QUESTIONS |
| **Information missing** | Add to Section 11 (Open Items) with a question; status → QUESTIONS |

| | |
|---|---|
| **Ambiguous** | Add to Section 11 (Open Items) with clarification needed; status → QUESTIONS |

**DO NOT invent data.** If the BRD says "Epic feed" but doesn't specify CSV vs JSON, that's an Open Item. If merge keys are not mentioned, that's an Open Item. Phase 01 can return QUESTIONS multiple times until all required fields are resolved.

**STEP 3 - Determine required phases**

Based on the requirements content, identify which downstream phases are needed:

| Requirement | Adds to required_phases |
|---|---|
| Gold layer transformations mentioned | `"06"` |
| Silver transformations or business logic mentioned | `"05"` |
| SQL views or Gensys/Genpro catalogs mentioned | `"07"` |
| Test scenarios or DQ rules mentioned | `"08"` |

**Always required (never conditional):** `"01"`, `"02"`, `"03"`, `"04"`, `"09"`, `"10"`

Include `required_phases` in the PHASE_RESULT (see Section 7.3).

**STEP 4 - Write requirements.md**

Use the canonical template at `adh-templates/artifacts/requirements.template.md`.

```
Output path:
.adh/workspace/output/{custodian}/{project}/requirements.md

Rules:
- Copy the template structure (core sections 0-11, optional sections
12-15 as needed)
- Fill all {{PLACEHOLDERS}} with actual values
- Delete all <!-- GUIDANCE ... --> comments
- Mark non-applicable sections "N/A - <reason>" rather than omitting
them
- Section 11 (Open Items) lists all conflicts, missing fields, and
ambiguities
- Fill required sections (marked [REQUIRED] in template) before status
can be APPROVED
```

```
- Use markdown formatting (headers, tables, lists)

```

**STEP 5 - Self-validation**

Run the validation skill:

```
py .github/skills/validate-config/validate_config.py \
   .adh/workspace/output/{custodian}/{project}/requirements.md \
   --check-template

```

**Expected outcomes:**

| Validation result | Status decision |
|---|---|
| PASS + Section 11 empty | `status: PASS` |
| PASS + Section 11 has items | `status: QUESTIONS` (surface each Open Item) |
| FAIL (missing required sections) | `status: FAIL` (report which [REQUIRED] sections are missing or unfilled) |

**STEP 6 - Update tracker (on PASS only)**

```
py .github/skills/tracker-checkpoint/tracker.py update \
   --custodian {custodian} \
   --project {project} \
   --phase 01 \
   --status done \
   --outputs requirements.md

```

**If FAIL or QUESTIONS:** Do NOT call this. Leave tracker status as `started`.

**STEP 7 - Return PHASE_RESULT**

See Section 7 for the full contract.

---

### 6. requirements.md Template

The specialist uses the **canonical template** at `adh-templates/artifacts/requirements.template.md`.

#### 6.1 Template structure

**File:** `adh-templates/artifacts/requirements.template.md`

**Core sections (1-11):**

- **1.** Business Context [REQUIRED]
- **2.** Scope [REQUIRED]
- **3.** Source System Details [REQUIRED]
- **4.** Load Strategy [REQUIRED]
- **5.** Target Schema [REQUIRED]
- **6.** Business Rules [REQUIRED]
- **7.** Data Quality Requirements [REQUIRED]
- **8.** Downstream Consumers [REQUIRED]
- **9.** SLA & Scheduling [REQUIRED]
- **10.** Non-Functional Requirements [REQUIRED]
- **11.** Open Items [REQUIRED - must be empty before APPROVED]

**Optional sections (12-15):**

- **12.** Assumptions [OPTIONAL]
- **13.** Acceptance Criteria [OPTIONAL - recommended]
- **14.** Traceability [OPTIONAL]
- **15.** Glossary [OPTIONAL]

#### 6.2 How the specialist uses the template

**Step 1 - Copy template structure:**

```
Read: adh-templates/artifacts/requirements.template.md
Output: .adh/workspace/output/{custodian}/{project}/requirements.md

```

**Step 2 - Fill placeholders:**

- Replace all `{{PLACEHOLDER}}` markers with actual values extracted from BRD/transcript/chat
- `{{CUSTODIAN_CODE}}` ← from delegation prompt (master provides via build_prompt.py)
- `{{PROJECT_OR_DATASET}}` ← from delegation prompt
- `{{YYYY-MM-DD}}` ← current date
- All other `{{X}}` ← from input sources

**Step 3 - Delete guidance comments:**

- Remove all `<!-- GUIDANCE: ... -->` comments
- Keep structural headers and tables

**Step 4 - Mark non-applicable sections:**

- If a section doesn't apply: write "N/A - " rather than omitting it
- Example: "**Gensys catalog:** N/A - no SQL views required for this project"

**Step 5 - Populate Section 11 (Open Items):**

- Add every conflict, ambiguity, or missing [REQUIRED] field as a row in the Section 11 table
- Each open item gets: Q# | Question | Owner | Due | Status
- **Critical rule:** Section 11 must be empty before requirements.md can be approved

**Step 6 - Include optional sections as needed:**

- Section 12 (Assumptions) - if assumptions were made
- Section 13 (Acceptance Criteria) - recommended for all projects
- Section 14 (Traceability) - if reverse-engineered from existing code
- Section 15 (Glossary) - if project-specific terms exist

#### 6.3 Template rules

1. **All [REQUIRED] sections must be filled.** Cannot be "N/A" or omitted.
2. **If information is missing:** Add to Section 11 (Open Items) with a specific question.
3. **If sources conflict:** Pick the most authoritative source (formal BRD > transcript > chat), but **also** add the conflict to Section 11 so the developer can confirm.
4. **Section 11 drives QUESTIONS status.** If Section 11 has items → `status: QUESTIONS` in PHASE_RESULT. Each item becomes a question the master asks the developer.
5. **Custodian and environment values come from the delegation prompt,** not from the BRD. The master provides these via `build_prompt.py`.
6. **Context pack references:** The template points to context-pack docs (project-context.md, source-systems.md, business-rules.md, etc.). If those don't exist yet for this project, note "To be created in Phase 02/03" in the relevant section.

---

### 7. PHASE_RESULT Contract

Every specialist agent **must** end its response with this structured block. The master parses it to decide next steps.

#### 7.1 PASS example (no open items)

```
## PHASE_RESULT
status: PASS
files_created:
  - path: .adh/workspace/output/core_ops/patient_data/requirements.md
    validated: true
```

```
    sections_filled: 12
    optional_sections: 1
validation_result: PASS - all [REQUIRED] sections filled, Section 11
(Open Items) empty
required_phases:
  - "01"
  - "02"
  - "03"
  - "04"
  - "05"
  - "06"
  - "09"
  - "10"
key_decisions:
  - load_type=INCREMENTAL
  - source_format=CSV
  - pii_present=Yes
tracker_updated: true

```

**Master action:** Proceed to Step E (approval gate for Phase 01).

#### 7.2 QUESTIONS example (open items exist)

```
## PHASE_RESULT
status: QUESTIONS
files_created:
  - path: .adh/workspace/output/core_ops/patient_data/requirements.md
    validated: true
    sections_filled: 12
    open_items: 3
validation_result: PASS with open items - Section 11 has 3 items
requiring clarification
questions:
  - id: Q1
    field: source_format
    context: "BRD mentions 'Epic feed' but does not specify format"
    message: "Is the source data format CSV, JSON, Parquet, or delta?"
  - id: Q2
    field: watermark_column
    context: "Load type is INCREMENTAL but no watermark column
mentioned"
    message: "What column tracks new or changed records in the source
table?"
  - id: Q3
    field: pii_columns
    context: "Section 4 notes PII present but does not list which
columns"
    message: "Which columns contain PII?"
required_phases: []
tracker_updated: false

```

**Master action:** Ask each question one at a time via `vscode/askQuestion`, collect answers, re-run `build_prompt.py ... --rework "Q1={answer}, Q2={answer}, Q3={answer}"`, and re-delegate to this specialist.

**Round-trip loop:** The specialist may return QUESTIONS again if answers are incomplete or ambiguous. No fixed limit. Developer is the circuit-breaker (can choose "Pause").

#### 7.3 FAIL example (template validation failed)

```
## PHASE_RESULT
status: FAIL
files_created:
  - path: .adh/workspace/output/core_ops/patient_data/requirements.md
    validated: false
validation_result: FAIL - Section 6 (Schema) is missing
required_phases: []
tracker_updated: false

```

**Master action:** Rework loop (Step F) - master shows the error, asks developer for guidance, re-delegates.

#### 7.4 Required fields in PHASE_RESULT

| Field | Type | Required? | Notes |
|---|---|---|---|
| `status` | String | Yes | One of: `PASS`, `FAIL`, `WARN`, `QUESTIONS` |
| `files_created` | Array | Yes | List of output files with validation status |
| `validation_result` | String | Yes | Human-readable summary from validate_config.py |
| `required_phases` | Array | Yes (if PASS) | List of phase IDs needed (e.g., `["01","02",...,"10"]`) |
| `questions` | Array | Yes (if QUESTIONS) | Structured questions for the developer |
| `tracker_updated` | Boolean | Yes | `true` only if `tracker.py update 01 done` ran successfully |

| | | | |
|---|---|---|---|
| `key_decisions` | Array | Optional | High-level values for session-context.json (e.g., load_type, source_format, pii_present) |

**Phase 01 does NOT populate** `lld_compliance` - that field is only for coding phases (04–06).

---

## 8. Context Management

#### 8.1 What to hold in context

**During execution:**

- BRD content (provided by master in delegation prompt)
- Transcript content
- Chat messages
- Rework feedback (if retrying)

**Do NOT hold:**

- Prior phase outputs (not applicable for Phase 01)
- Full tracker JSON (only read current_phase)
- Custodian config (master resolves it via build_prompt.py)

#### 8.2 What to release after completion

Once PHASE_RESULT is returned, the specialist's turn ends. The master takes over. The specialist does not persist any state beyond what's written to `requirements.md` and `project-tracker.json`.

**Compact delegation:** The master passes a fully-resolved prompt (no `{placeholders}`). The specialist processes it once and returns. No multi-turn conversation within the specialist.

---

## 9. Rework Handling

When the master re-delegates to this specialist with `--rework "{feedback}"`:

**STEP 1 - Read prior requirements.md**

```
path: .adh/workspace/output/{custodian}/{project}/requirements.md
```

```

Read the existing file to understand what was previously written.

```

**STEP 2 - Parse rework feedback**

Feedback format examples:

- `"Q1=CSV, Q2=updated_at, Q3=patient_id,ssn,dob"` (answering QUESTIONS)
- `"Change load_type to FULL"` (freeform developer guidance)
- `"Add SLA: data available by 9 AM ET daily"` (new requirement)

**STEP 3 - Update requirements.md**

Incorporate the feedback into the appropriate section(s). Do NOT rewrite the entire file unless necessary. Update only the sections that need changes.

**STEP 4 - Re-validate**

Run `validate_config.py` again. If new Open Items emerge, return QUESTIONS again. If all resolved, return PASS.

**Rework is not failure.** Rework is a normal part of the pipeline. Phase 01 may go through multiple QUESTIONS rounds before all information is gathered.

---

### 10. Hard Rules (in the instruction body)

These rules **cannot be overridden** by any user request, master instruction, or delegation prompt:

```
HARD RULES:
1. I do NOT ask questions directly to the developer. I surface questions via PHASE_RESULT.
2. I do NOT invent missing data. I flag it as an Open Item (Section 11).
3. I do NOT skip or omit [REQUIRED] sections in requirements.md. All must be filled.
4. I do NOT resolve conflicts by guessing. I add both versions to Section 11 and ask.
5. I do NOT invoke other specialist agents. I work alone.
6. I do NOT proceed to Phase 02. The master handles phase transitions.
7. I do NOT edit files in .adh/tracker/ directly. I only call tracker.py scripts.
8. I do NOT write files outside .adh/workspace/output/{custodian}/{project}/.
9. I update the tracker to "started" before any work, and to "done" only on PASS.
10. If validation fails, I return FAIL - never silently fix and retry.
11. I ALWAYS use the canonical template at adh-templates/artifacts/requirements.template.md.

```

**Why these rules?** They enforce the orchestrated pipeline model: the master coordinates, the specialists execute narrowly-scoped tasks, and the developer makes all judgment calls.

---

### 11. Presentation & UX

The specialist **does not** present Phase Cards, progress trackers, or status symbols. That's the master's job (see `ADH-Pilot-Master-Agent-TechSpec.md`, Section 13).

**What the specialist outputs:**

1. **Diagnostic messages (optional):**
   - "Reading BRD file: {path}"
   - "Extracting requirements from {N} sources"
   - "Validation passed - no open items"
   - "Validation passed - 3 open items found"
2. **PHASE_RESULT block (mandatory):** Always the last output.

**No emoji, no decorative formatting.** Clean, factual prose. The master handles all UX polish.

---

### 12. Acceptance Criteria

| AC | Test Scenario | Expected Result |
|---|---|---|
| **AC-01** | Single BRD, all required fields present | `status: PASS`, all [REQUIRED] sections filled, Section 11 empty, required_phases populated |
| **AC-02** | BRD missing source_format | `status: QUESTIONS`, Q1 asks for source_format, Section 11 has 1 item |
| **AC-03** | BRD has conflicting load_type (says FULL in one place, INCREMENTAL in another) | `status: QUESTIONS`, Q1 asks which is correct, Section 11 lists both |

| | | |
|---|---|---|
| **AC-04** | BRD + transcript + 2 chat messages | All sources merged, conflicts flagged, PASS or QUESTIONS as appropriate |
| **AC-05** | Rework with answers to 2 questions | requirements.md updated, Section 11 cleared, `status: PASS` |
| **AC-06** | Rework with incomplete answer | New QUESTIONS returned with refined question |
| **AC-07** | BRD mentions Gold layer | `required_phases` includes `"06"` |
| **AC-08** | BRD mentions SQL views for Gensys/Genpro | `required_phases` includes `"07"` |
| **AC-09** | BRD has no Silver or Gold transformations | `required_phases` excludes `"05"` and `"06"` |
| **AC-10** | Validation fails (Section 5 Target Schema not filled) | `status: FAIL`, validation_result explains which [REQUIRED] section is missing/unfilled |
| **AC-11** | Phase guard failure (current_phase = "02") | `status: FAIL`, message="Phase guard failed - expected 01, got 02" |
| **AC-12** | Tracker update to "started" | `project-tracker.json` phase 01 `status=started`, |

| | | |
|---|---|---|
| | | `started_at` populated |
| **AC-13** | Tracker update to "done" (on PASS) | `project-tracker.json` phase 01 `status=done`, `completed_at` populated, `outputs=["requirements.md"]` |
| **AC-14** | Tracker not updated (on FAIL or QUESTIONS) | `project-tracker.json` phase 01 `status=started` (unchanged) |
| **AC-15** | Multiple QUESTIONS rounds | Each round returns structured questions; Section 11 shrinks as items are resolved |
| **AC-16** | Developer provides freeform rework feedback | Specialist parses it, updates relevant sections, re-validates |
| **AC-17** | Output file isolation | File written to `.adh/workspace/output/{custodian}/{project}/requirements.md` only |